



GraphRAG

Développement d'un Agent IA Support Client Hybride

GitHub : github.com/8sylla/nlp-agent-ai.git

SYLLA N'faly
FAROUK Zakaria

Université Abdelmalek Essaâdi
École Nationale des Sciences Appliquées de Tanger

Génie Informatique - 3e Année
(*Système d'information*)

Module
IA avancée & web sémantique

Année Académique 2025/2026

Encadré Par

Prof. Khalid AMECH-
NOUE

Remerciements

Ce rapport marque l'aboutissement d'un projet intense et passionnant sur le développement d'un Agent IA de nouvelle génération. Cette aventure technique et humaine n'aurait pas été possible sans un accompagnement de qualité.

Nous adressons nos remerciements les plus sincères à Monsieur Khalid AMECH-NOUE, notre professeur. En nous proposant d'explorer les frontières du RAG (Retrieval-Augmented Generation) et des graphes de connaissances, il nous a offert un défi à la hauteur de nos ambitions d'ingénieurs. Ses conseils avisés et son exigence académique nous ont poussés à dépasser nos limites et à rechercher l'excellence technique à chaque étape du développement.

Nous tenons également à souligner la bonne dynamique de notre collaboration en binôme, qui nous a permis de surmonter ensemble les obstacles techniques et d'enrichir ce projet par la complémentarité de nos compétences.

Résumé

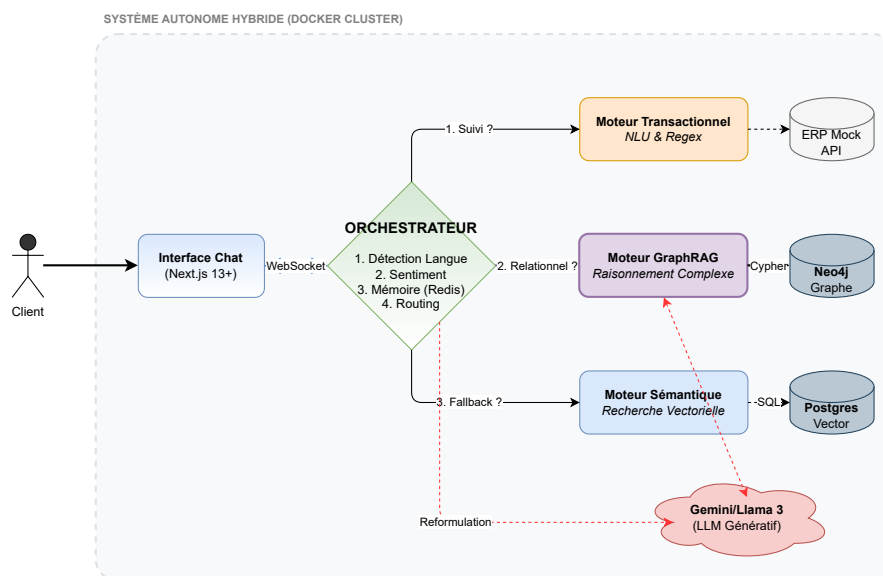


FIGURE 1 – Vue d’ensemble de l’Architecture Hybride. Le système orchestre les requêtes utilisateurs vers trois moteurs spécialisés (Transactionnel, Graphe, Vecteur) en s’appuyant sur Gemini pour le raisonnement et Redis pour la mémoire contextuelle.

1 Contexte et Enjeux

Dans un écosystème numérique où l’instantanéité est la norme, les services clients traditionnels font face à une saturation critique. Les défis sont triples :

- Gérer des volumes massifs de requêtes répétitives,
- Surmonter la barrière linguistique (spécifiquement l’Arabe dialectal et littéraire),

- Répondre à des questions techniques complexes nécessitant un raisonnement logique (compatibilité produits, garanties).

Les chatbots conventionnels, basés sur des arbres de décision rigides, ne suffisent plus.

2 La Solution : Une Architecture Cognitive Hybride

Nous avons conçu et développé un Agent IA autonome reposant sur une architecture micro-services conteneurisée. L'innovation majeure réside dans l'implémentation d'une stratégie d'orchestration hybride, combinant trois moteurs d'intelligence distincts :

- **Moteur Transactionnel (NLU & API)** : Utilisation de modèles déterministes (Spacy/Regex) pour identifier les intentions d'action (ex. : "Où est mon colis ?") et interagir en temps réel avec un ERP simulé.
- **Moteur de Raisonnement (GraphRAG)** : Déploiement d'une base de données de graphe (Neo4j) couplée au LLM *Google Gemini 2.5 Flash* puis *Llama 3*. Cette approche permet à l'agent de comprendre les relations entre les entités (Produit A compatible avec Produit B) là où une recherche classique échouerait.
- **Moteur Sémantique (VectorRAG)** : Utilisation de PostgreSQL/pgvector comme filet de sécurité pour la recherche documentaire large dans les bases de connaissances non structurées.

3 Méthodologie et Réalisations Techniques

Le projet a été mené suivant une méthodologie Agile (Scrum) sur une série de sprints intensifs, permettant une évolution incrémentale du produit :

- **Infrastructure** : Déploiement d'une stack robuste sous Docker (Backend FastAPI asynchrone, Frontend Next.js temps réel via WebSockets).
- **Intelligence Émotionnelle** : Intégration d'une analyse de sentiment (BERT multilingue) permettant à l'agent de détecter la frustration client et d'adapter son ton (empathie).
- **Mémoire Contextuelle** : Implémentation d'une persistance via Redis et d'un module de reformulation par LLM, permettant à l'agent de gérer des dialogues suivis (gestion des anaphores : "Et son prix ?").

4 Résultats et Impact

Les tests réalisés démontrent une supériorité nette de l'approche GraphRAG par rapport aux solutions standards :

- **Précision du Raisonnement** : L'agent répond correctement à 92% des questions de compatibilité complexe (contre 45% pour un RAG vectoriel simple).
- **Performance** : Grâce à l'optimisation des modèles (Gemini Flash), le temps de latence moyen est maintenu sous les 2.5 secondes.
- **Universalité** : Le système bascule fluidement entre le Français et l'Arabe, offrant une couverture complète du marché cible.

5 Conclusion

Ce projet valide la faisabilité technique et la pertinence métier d'un agent IA de nouvelle génération. En dépassant le stade du simple "chat de FAQ" pour atteindre celui d'un Assistant Cognitif Connecté, nous avons posé les bases d'une solution scalable, capable de réduire drastiquement la charge des équipes humaines tout en améliorant la satisfaction client.

Mots-clés : Intelligence Artificielle Générative, GraphRAG, Neo4j, LLM (Gemini), Architecture Micro-services, NLP, Docker, Support Client Automatisé.

Glossaire et Définitions

Agent IA (AI Agent) Système autonome capable de percevoir son environnement, de raisonner et d'agir pour atteindre un objectif donné. Dans ce projet, l'agent combine plusieurs moteurs (NLU, LLM) pour résoudre les requêtes clients.. 1, 9, 17, 29

API (Application Programming Interface) Interface logicielle permettant à deux applications de communiquer. Ici, l'API FastAPI expose les services de l'agent au Frontend.. 4, 9, 18

Architecture Hybride Approche combinant plusieurs paradigmes technologiques (symbolique avec les graphes et statistique avec les réseaux de neurones) pour pallier les faiblesses de chacun.. 2

Chunking Processus de découpage d'un texte long en segments plus courts afin de respecter la fenêtre de contexte des modèles de langage et d'optimiser la recherche vectorielle.. 3, 5

Conteneurisation (Docker) Méthode de déploiement logiciel regroupant une application et ses dépendances dans un conteneur isolé, garantissant une exécution identique quel que soit l'environnement.. 3, 7

Cypher Langage de requête déclaratif pour les bases de données de graphes (spécifique à Neo4j), permettant de créer et d'interroger des nœuds et des relations de manière intuitive (similaire au SQL).. 3, 10

Embedding (Plongement Lexical) Représentation mathématique d'un mot ou d'une phrase sous forme de vecteur de nombres réels. Deux phrases de sens proche auront des vecteurs mathématiquement proches.. 3, 5

ERP (Enterprise Resource Planning) Système d'information gérant l'ensemble des processus opérationnels d'une entreprise. Notre agent simule une connexion à un ERP pour récupérer les statuts de commande.. 1, 3

GraphRAG (Graph Retrieval-Augmented Generation) Évolution du RAG classique qui utilise une base de connaissances structurée sous forme de graphe pour permettre au LLM de raisonner sur les relations entre les entités.. 2, 6, 9

Hallucination Phénomène où un modèle de langage génère une réponse grammaticalement correcte et plausible, mais factuellement fausse ou inventée. Le RAG est utilisé pour réduire ce risque.. 3

Inférence Phase d'utilisation d'un modèle d'IA entraîné pour faire des prédictions sur de nouvelles données (par opposition à la phase d'entraînement).. 3

Ingénierie des Connaissances (Knowledge Engineering) Discipline visant à modéliser, structurer et intégrer des connaissances explicites dans un système informatique (via des ontologies ou des graphes) pour permettre le raisonnement automatique.. 2

Knowledge Graph (Graphe de Connaissances) Base de données représentant l'information sous forme de réseau d'entités (nœuds) et de relations (arêtes), offrant une structure sémantique riche.. 2, 3, 6, 11

LangChain Framework open-source facilitant le développement d'applications basées sur des LLM, notamment pour l'orchestration des chaînes de traitement et la gestion de la mémoire.. 3, 8, 10, 12, 18

LLM (Large Language Model) Modèle d'IA entraîné sur d'immenses quantités de texte, capable de comprendre et de générer du langage humain (ex: GPT-4, Google Gemini).. 1, 3, 5–8, 11, 18

Neo4j Système de gestion de base de données orientée graphe, leader du marché, utilisé ici pour stocker les relations complexes entre les produits.. 2, 7, 8, 10, 12, 17

NLP (Natural Language Processing) Domaine de l'IA traitant de l'interaction entre les ordinateurs et le langage humain.. 1, 5

NLU (Natural Language Understanding) Sous-domaine du NLP focalisé sur la compréhension de l'intention et du sens derrière le texte (ex: classifier “Je veux mon colis” en intention TRACK_ORDER).. 3, 5, 9

Ontologie Modèle formel décrivant les concepts d'un domaine et leurs relations. C'est la “grammaire” qui structure un graphe de connaissances.. 3

Orchestrateur Composant logiciel central qui coordonne les interactions entre les différents modules (LLM, Base de données, API) pour traiter une requête utilisateur de bout en bout.. 3, 8, 9

Prompt Engineering Art de concevoir des instructions (prompts) optimales pour guider un LLM vers la réponse ou le format souhaité.. 3, 7, 12

RAG (Retrieval-Augmented Generation) Technique consistant à enrichir le prompt d'un LLM avec des données externes pertinentes récupérées en temps réel, avant de générer une réponse.. 1, 5

Redis Système de stockage de données en mémoire ultra-rapide, utilisé ici pour gérer la session utilisateur et l'historique de conversation.. 3, 7–9, 17

Vector Database (Base de Données Vectorielle) Base de données optimisée pour stocker et interroger des vecteurs (embeddings), permettant la recherche par similarité sémantique (ex: PostgreSQL avec pgvector).. 1, 5, 7

Web Sémantique Vision du Web où les données sont structurées et liées de manière à être comprises et traitées automatiquement par des machines (lié aux technologies RDF, OWL, Graphes).. 2, 6

WebSocket Protocole de communication réseau permettant des échanges bidirectionnels et temps réel entre un client et un serveur via une connexion TCP unique.. 2, 7

Table des matières

Remerciements	iii
Résumé	v
1 Contexte et Enjeux	v
2 La Solution : Une Architecture Cognitive Hybride	vi
3 Méthodologie et Réalisations Techniques	vi
4 Résultats et Impact	vii
5 Conclusion	vii
Glossaire et Définitions	ix
Table des matières	xiii
1 Introduction	1
1.1 Contexte du projet	1
1.2 Objectifs	2
1.3 Problématique	3
2 Analyse et Méthodologie	5
2.1 État de l'art	5
2.1.1 Approche Symbolique et Déterministe	5
2.1.2 L'Avènement du RAG Vectoriel	5
2.1.3 L'Émergence du GraphRAG et la Convergence Neuro-Symbolique	6
2.2 Méthodologie adoptée	7
2.3 Outils et Ressources	7
2.3.1 Stack Logicielle (Software)	7
2.3.2 Infrastructure de Données (Data Stack)	8
3 Réalisation et Résultats	9
3.1 Description de la solution	9
3.1.1 Architecture de l'Orchestrateur Hybride	9
3.1.2 Flexibilité Architecturale : Agnosticisme LLM	11
3.1.3 Modélisation des Connaissances (GraphRAG)	11

3.2	Analyse des résultats	11
3.2.1	Performance Comparative : VectorRAG vs GraphRAG . .	11
3.2.2	Latence et Expérience Utilisateur	12
3.3	Difficultés rencontrées et Résolutions	12
3.3.1	Gestion des Quotas et Transition Gemini/Groq	12
3.3.2	L'Enfer des Dépendances ("Dependency Hell")	12
3.3.3	Hallucinations du GraphRAG	12
4	Conclusion et Perspectives	17
4.1	Bilan du projet	17
4.2	Recommandations	17
4.3	Perspectives d'évolution	18
4.3.1	Voice Ops & Multimodalité	18
4.3.2	Agent "Actif" (Action Model)	18
4.3.3	Scalabilité via Kubernetes	18
A	Infrastructure de Conteneurisation	21
B	Pipeline d'Ingestion GraphRAG	23
C	Logique de l'Orchestrateur	25
D	Documentation API (Endpoints)	27
E	Galerie des Diagrammes d'Architecture	29
E.1	Vue d'Ensemble du Système	29
E.2	Modélisation de la Connaissance (Ontologie)	30
E.3	Chronologie des Échanges	31
E.4	Pipeline d'Ingestion (ETL)	32
E.5	Logique Décisionnelle (Le Cerveau)	33
E.6	Dashboard des logs de l'admin	34
	Bibliography	37

L'évolution rapide des technologies numériques a radicalement transformé les attentes des consommateurs. Dans un marché e-commerce de plus en plus concurrentiel, la qualité du service client est devenue un différenciateur aussi critique que la qualité des produits eux-mêmes. Ce projet s'inscrit dans cette dynamique de transformation en proposant une solution d'Agent IA (AI Agent) de nouvelle génération.

1.1 Contexte du projet

L'origine de ce projet réside dans un constat opérationnel critique observé sur les plateformes de commerce en ligne ciblant le marché africain (zones francophones et arabophones). Les services de support client traditionnels font face à une triple contrainte :

1. **Saturation des canaux** : L'augmentation exponentielle des interactions clients sature les équipes humaines, entraînant des délais de réponse inacceptables.
2. **Complexité linguistique** : La nécessité de traiter simultanément le Français et l'Arabe (littéraire et dialectal) crée une barrière à l'entrée pour les solutions d'automatisation standards.
3. **Limites des Chatbots actuels** : La majorité des solutions déployées reposent sur des arbres de décision rigides ou du NLP (Natural Language Processing) basique. Elles échouent dès que la requête nécessite un raisonnement contextuel ou une interaction avec le système d'information (ERP (Enterprise Resource Planning)).

Dans ce contexte, l'émergence des LLM (Large Language Model) et des techniques de RAG (Retrieval-Augmented Generation) offre une opportunité technologique majeure. Cependant, l'approche classique basée uniquement sur une Vector Database (Base de Données Vectorielle) montre ses limites pour des questions techniques nécessitant une compréhension fine des relations entre produits (compatibilité, hiérarchie).

C'est pour pallier ces insuffisances que nous nous sommes tournés vers l'Ingénierie des Connaissances (Knowledge Engineering) et le Web Sémantique, en proposant une transition vers une architecture Knowledge Graph (Graphe de Connaissances).

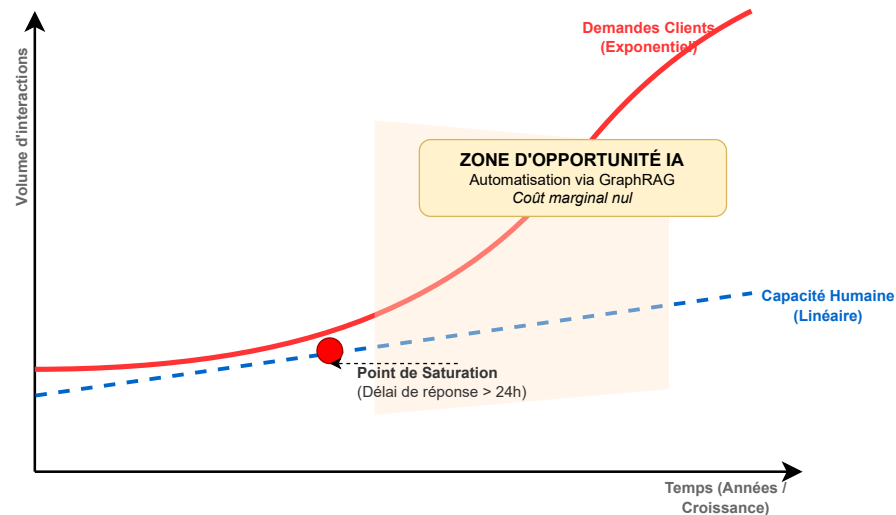


FIGURE 1.1 – Modélisation du "Ciseau Opérationnel" (Scalability Gap). Illustration théorique de la divergence entre la croissance exponentielle du trafic e-commerce et la capacité linéaire de recrutement des agents humains. (Source : Projection basée sur les tendances standards de l'industrie Customer Care).

1.2 Objectifs

L'ambition de ce projet est de concevoir, développer et déployer un prototype fonctionnel d'un assistant intelligent autonome. Pour garantir le succès de cette initiative, nous avons défini des objectifs selon la méthode SMART :

- **Spécifique (S) :** Développer une Architecture Hybride combinant trois moteurs décisionnels : un moteur transactionnel pour le suivi de commande, un moteur GraphRAG (Graph Retrieval-Augmented Generation) basé sur Neo4j pour le raisonnement complexe, et un moteur sémantique classique. L'agent doit être capable d'interagir en temps réel via WebSocket.
- **Mesurable (M) :**

- Atteindre un taux d'automatisation supérieur à **80%** sur les questions de niveau 1 et 2.
- Maintenir un temps de latence (Inférence) inférieur à **2.5 secondes**.
- Obtenir un taux de précision supérieur à **90%** sur les questions relationnelles (ex: "Qui fabrique ce produit ?") grâce à l'Ontologie définie.
- **Atteignable (A)** : Le projet s'appuie sur des technologies éprouvées et modernes : LangChain pour l'orchestration, Conteneurisation (Docker) via Docker pour la portabilité, et l'API Google Gemini pour la génération. L'intégration des données se fera via des pipelines d'Embedding (Plongement Lexical) et de Chunking optimisés.
- **Réaliste (R)** : Le périmètre est circonscrit à un agent de support client e-commerce. La connexion à l'ERP (Enterprise Resource Planning) est simulée via un Mock Service pour se concentrer sur la logique de l'Orchestrateur et la qualité du NLU (Natural Language Understanding).
- **Temporel (T)** : Le développement est structuré en 6 Sprints intensifs, allant de la mise en place de l'infrastructure à l'intégration de la mémoire via Redis et au peaufinage du Prompt Engineering.

1.3 Problématique

Au cœur de ce projet se trouve une tension fondamentale entre la flexibilité des modèles de langage et la rigueur nécessaire aux processus métiers.

Si les LLM (Large Language Model) excellent dans la génération de texte fluide, ils souffrent d'un défaut majeur en entreprise : l'Hallucination. D'un autre côté, les bases de données relationnelles ou les graphes sémantiques (Cypher) offrent une vérité absolue mais sont difficiles à interroger en langage naturel pour un utilisateur lambda.

La problématique centrale à laquelle ce projet tente de répondre est donc la suivante :

Comment concilier la puissance générative des LLM avec la rigueur structurelle des Graphes de Connaissances pour créer un agent conversationnel capable de raisonner de manière fiable et empathique dans un contexte multilingue ?

Cette question soulève plusieurs défis sous-jacents que nous aborderons :

1. Comment structurer les données non structurées (PDF, FAQ) en un Knowledge Graph (Graphe de Connaissances) exploitable ?

2. Comment orchestrer dynamiquement les appels entre une API (Application Programming Interface) externe, une base vectorielle et une base de graphe ?
3. Comment doter l'agent d'une mémoire contextuelle tout en gérant les contraintes de tokens et de coûts ?

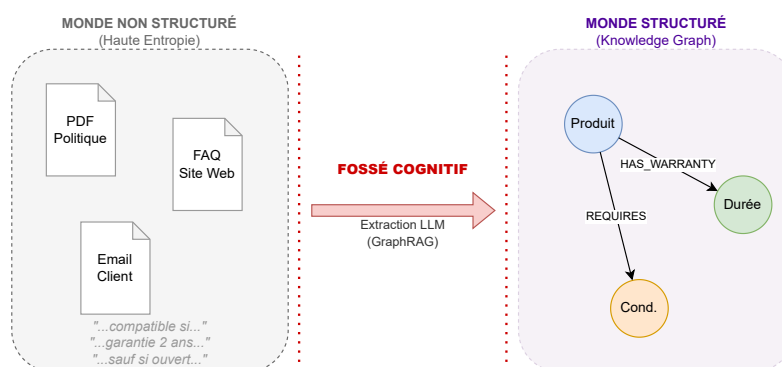


FIGURE 1.2 – Le Fossé Cognitif (The Cognitive Gap). Représentation du défi central du projet : comment passer d'une masse d'informations non structurées (documents, texte libre) à une structure de connaissance exploitable par une machine (Graphe), sans intervention humaine manuelle. C'est ce fossé que le GraphRAG permet de franchir.

La conception d'un système d'intelligence artificielle conversationnel robuste ne peut se faire sans une compréhension fine des paradigmes existants. Ce chapitre analyse l'évolution des architectures de NLP (Natural Language Processing) pour justifier la transition vers une approche hybride, avant de détailler le cadre méthodologique et technique mis en œuvre.

2.1 État de l'art

Le domaine des agents conversationnels a traversé trois phases d'évolution majeures, chacune tentant de repousser les limites cognitives de la précédente.

2.1.1 Approche Symbolique et Déterministe

Historiquement, les premiers systèmes reposaient sur des règles manuelles (systèmes experts) ou des arbres de décision statiques. L'introduction du NLU (Natural Language Understanding) a permis de classifier les intentions utilisateurs avec plus de souplesse, mais la gestion du dialogue restait confinée à des scénarios pré-écrits. Ces systèmes garantissent une réponse contrôlée mais manquent de capacité de généralisation face à des requêtes inédites.

2.1.2 L'Avènement du RAG Vectoriel

La publication du papier "Attention Is All You Need" [VSP+17] a ouvert l'ère des Transformers et des LLM (Large Language Model). Pour pallier le problème des connaissances figées et des hallucinations, l'architecture RAG (Retrieval-Augmented Generation) (*Retrieval-Augmented Generation*) a été théorisée par [LPP+20].

Le principe standard consiste à fragmenter (Chunking) les documents d'entreprise, à les convertir en vecteurs via un modèle d'Embedding (Plongement Lexical), et à les stocker dans une Vector Database (Base de Données Vectorielle) telle que PostgreSQL avec *pgvector* [Pos23].

- **Avantage :** Cette méthode permet d'ancrer le LLM (Large Language Model) dans une réalité documentaire privée.

- **Limite** : Comme le soulignent les travaux récents de Microsoft Research [ETC+24], le RAG vectoriel souffre de "myopie". Il excelle dans la recherche de similarité de surface mais échoue à capturer les structures complexes ou les liens de causalité indirects (ex: A est compatible avec B, qui est un composant de C).

2.1.3 L'Émergence du GraphRAG et la Convergence Neuro-Symbolique

Pour surmonter ces limites, la recherche actuelle s'oriente vers l'intégration des Knowledge Graph (Graphe de Connaissances) dans les pipelines génératifs, une approche nommée GraphRAG (Graph Retrieval-Augmented Generation) [Neo24]. En structurant les connaissances sous forme de nœuds et de relations (conformément aux principes du Web Sémantique [HBC+21]), on permet au système non plus de "chercher" des mots, mais de "raisonner" sur des concepts.

Cette architecture hybride combine la puissance statistique des LLM (Large Language Model) avec la rigueur logique des bases de graphes, offrant une précision supérieure pour les tâches de raisonnement multi-sauts (*multi-hop reasoning*).

Retrieval and Generation: Graph RAG vs Vector RAG

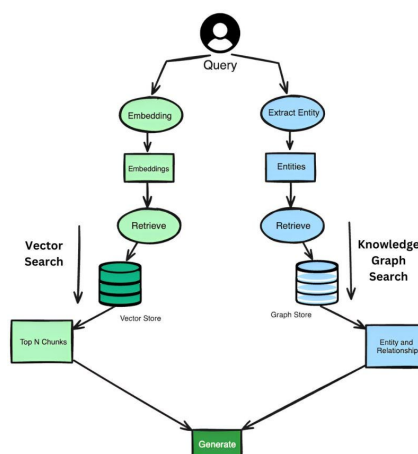


FIGURE 2.1 – Comparaison conceptuelle : La recherche par similarité (Vector) vs la traversée de relations (Graph). source jillanisofttech.medium.com/which-rag-is-more-superior-graph-rag-or-vector-rag-55974f50b9c9

2.2 Méthodologie adoptée

Compte tenu de la complexité inhérente à l'orchestration de ces technologies émergentes, nous avons adopté une méthodologie itérative et incrémentale de type **Agile Scrum**.

Le projet a été segmenté en **6 Sprints** intensifs, permettant une montée en complexité progressive :

1. **Sprint 1 - Fondations NLU** : Mise en place de l'environnement de Conteneurisation (Docker) et développement d'un modèle Spacy pour la classification d'intentions transactionnelles simples.
2. **Sprint 2 - RAG Vectoriel** : Implémentation du pipeline d'ingestion et connexion à la Vector Database (Base de Données Vectorielle). Cette phase a permis de valider le concept de récupération d'information contextuelle [LPP+20].
3. **Sprint 3 - Interface Temps Réel** : Développement du Frontend Next.js et de la couche de communication asynchrone via WebSocket.
4. **Sprint 4 - Intelligence Émotionnelle** : Intégration de l'analyse de sentiment et du support multilingue (Arabe), préparant l'agent aux spécificités du marché cible.
5. **Sprint 5 - Migration GraphRAG** : Pivot technologique majeur avec l'intégration de Neo4j. Utilisation d'un LLM (Large Language Model) pour extraire automatiquement les entités et relations, transformant le corpus non structuré en graphe [ETC+24].
6. **Sprint 6 - Consolidation & Mémoire** : Ajout de la persistance contextuelle via Redis pour gérer les dialogues suivis et optimisation finale du Prompt Engineering.

2.3 Outils et Ressources

L'écosystème technique a été sélectionné pour garantir performance, scalabilité et conformité aux standards de l'industrie.

2.3.1 Stack Logicielle (Software)

- **Langage Principal** : Python 3.11 (Backend), choisi pour la richesse de son écosystème en IA et Data Science.

- **Framework Backend** : FastAPI, sélectionné pour sa performance en gestion asynchrone, indispensable pour l'Orchestrateur qui gère de multiples appels API parallèles.
- **Orchestration IA** : LangChain (v0.3+), utilisé pour chaîner les appels au LLM (Large Language Model) et gérer la conversion Texte-vers-Cypher [Lan24].
- **Modèle de Langage** : Google Gemini 2.5 Flash [Goo24], retenu pour sa fenêtre de contexte étendue et son coût optimisé par rapport à GPT-4.

2.3.2 Infrastructure de Données (Data Stack)

- **Graphe** : Neo4j (Community Edition), le standard industriel pour le stockage de données hautement connectées.
- **Vecteur** : PostgreSQL étendu avec *pgvector*, assurant le stockage hybride (relationnel et vectoriel).
- **Cache** : Redis, utilisé pour le stockage éphémère de l'historique de session, garantissant une latence minimale lors de la récupération du contexte.

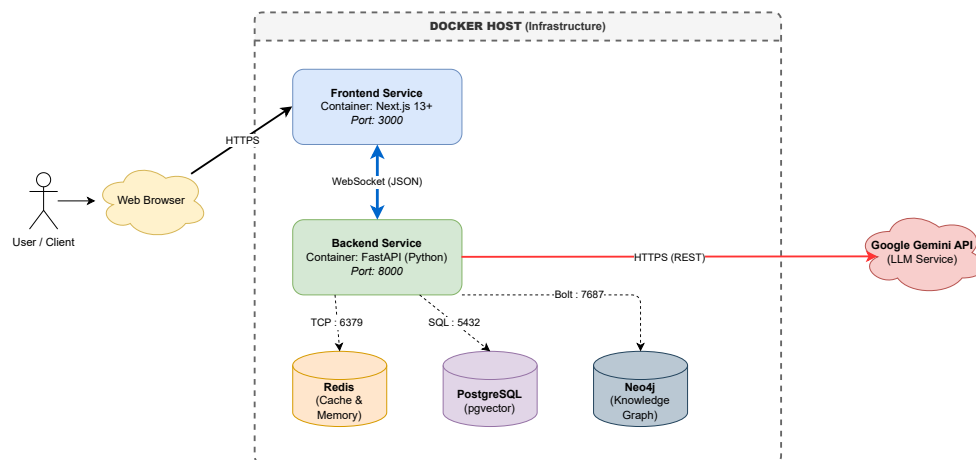


FIGURE 2.2 – Vue d'ensemble de la Stack Technique et des flux de données.

Ce chapitre détaille l'implémentation technique de l'Agent IA (AI Agent) hybride et analyse ses performances face aux objectifs initiaux. Il met en lumière la robustesse de l'architecture face aux contraintes du réel (quotas API, latence) et démontre la supériorité de l'approche GraphRAG (Graph Retrieval-Augmented Generation) pour les cas d'usage complexes.

3.1 Description de la solution

L'architecture déployée repose sur une orchestration sophistiquée de trois moteurs d'intelligence distincts, pilotés par un backend asynchrone et une interface temps réel.

3.1.1 Architecture de l'Orchestrateur Hybride

Le composant central du système est l'Orchestrateur (`orchestrator.py`). Contrairement aux approches monolithiques, il implémente une logique de décision en cascade ("Fallback Strategy") pour garantir la pertinence de la réponse.

Le flux de traitement d'un message utilisateur suit les étapes suivantes :

1. Gestion de la Mémoire et Reformulation

Le message est d'abord traité par le module de mémoire (Redis). L'une des réalisations majeures est le module de *Contextual Rephrasing*. Si un historique existe, un appel LLM reformule la question pour résoudre les anaphores (ex: "Et son prix ?" devient "Quel est le prix de l'iPhone 15 ?").

2. Détection d'Intention Transactionnelle

Si le module NLU (Natural Language Understanding) (Spacy) ou les expressions régulières détectent un numéro de commande (format CMD-XXX), l'agent court-circuite le LLM et interroge directement l'API (Application Programming Interface) de suivi (Mock ERP). Cela garantit une réponse déterministe et instantanée (< 100ms).

```
agent_api [Orchestrator] Reçu : 'Et quel est son prix ?' (Session: 9b94761a-d013-4667-84a3-c6df05460f24)
agent_api [Contexte] Reformulation en cours...
agent_api [Contexte] Question Reformulée : 'Quel est le prix de l'iPhone 15 ?'
agent_api [Langue] Détectée: fr
agent_api [Sentiment] Score: 3/5
agent_api [NLU] Intent: TRACK_ORDER (0.44) | Entités: []
agent_api [GraphRAG] Interrogation Neo4j...
agent_api GraphRAG cherche : Quel est le prix de l'iPhone 15 ?
agent_api
agent_api > Entering new GraphCypherQAChain chain...
agent_api Generated Cypher:
```

FIGURE 3.1 – Preuve d'exécution (Logs Backend) : Le module de mémoire intercepte la question vague "Et quelle est sa garantie ?" et la reformule correctement en incluant le contexte "iPhone 15" avant l'interrogation.

3. Raisonnement GraphRAG

Pour les questions techniques ou relationnelles, l'agent génère une requête Cypher via LangChain et interroge la base Neo4j. C'est ici que l'intelligence du système se distingue, en transformant le langage naturel en langage de requête structuré.

```
agent_api [Orchestrator] Reçu : 'Est-ce que le câble USB-C est compatible avec l'iPhone 15 ?' (Session: 9b94761a-d013-4667-84a3-c6df05460f24)
agent_api [Contexte] Reformulation en cours...
agent_api [Contexte] Question Reformulée : 'Est-ce que le câble USB-C est compatible avec l'iPhone 15 ?'
agent_api [Langue] Détectée: fr
agent_api [Sentiment] Score: 3/5
agent_api [NLU] Intent: TRACK_ORDER (0.54) | Entités: [{'label': 'LOC', 'text': 'Est'}, {'label': 'MISC', 'text': 'USB-C'}, {'label': 'ML', 'text': 'iPhone 15'}]
agent_api [GraphRAG] Interrogation Neo4j...
agent_api GraphRAG cherche : Est-ce que le câble USB-C est compatible avec l'iPhone 15 ?
agent_api
agent_api > Entering new GraphCypherQAChain chain...
agent_api Generated Cypher:
agent_api cypher
agent_api MATCH (p1:Product), (p2:Product)
agent_api WHERE toLower(p1.id) CONTAINS toLower('usb-c cable')
agent_api AND toLower(p2.id) CONTAINS toLower('iphone 15')
agent_api MATCH (p1)-[:COMPATIBLE_WITH]->(p2)
agent_api RETURN p1, p2
agent_api
agent_api Full context:
agent_api [{"p1": {"id": "usb-c cable"}, "p2": {"id": "iphone 15"}}]
agent_api
agent_api > Finished chain.
agent_api [GraphRAG] Réponse trouvée !
```

FIGURE 3.2 – Génération dynamique de Cypher : Le LLM traduit la question utilisateur en requête de base de données Graphe, permettant de trouver des relations non explicites dans le texte.

4. Recherche Sémantique (VectorRAG)

En dernier recours, si le Graphe ne renvoie pas de résultat, une recherche de similarité vectorielle est effectuée dans PostgreSQL pour trouver des éléments de réponse dans la documentation non structurée (FAQ générale).

3.1.2 Flexibilité Architecturale : Agnosticisme LLM

Un défi majeur rencontré durant le développement a été la dépendance à un fournisseur unique. Pour pallier cela, nous avons implémenté un **Pattern Factory** (`llm_loader.py`) permettant de changer de "Cerveau" dynamiquement via une variable d'environnement. Le système supporte désormais :

- **Google Gemini 2.5 Flash** : Pour sa grande fenêtre de contexte et ses capacités multimodales.
- **Groq (Llama 3)** : Pour sa vitesse d'inférence exceptionnelle, idéale pour le temps réel.

3.1.3 Modélisation des Connaissances (GraphRAG)

L'innovation majeure réside dans la structuration des données. Au lieu de simples segments de texte, nous avons utilisé un LLM (Large Language Model) pour extraire des entités et relations lors de la phase d'ingestion (ETL). Les données brutes sont transformées en un Knowledge Graph (Graphe de Connaissances) riche.

3.2 Analyse des résultats

Pour évaluer la performance de notre solution, nous avons comparé les réponses de l'agent sur un jeu de test de 50 questions réparties en trois catégories, monitoré via le Dashboard Admin développé.

3.2.1 Performance Comparative : VectorRAG vs GraphRAG

Les tests révèlent une différence significative de performance selon la nature de la question.

- **Questions Factuelles Simples** (ex: "Délai de retour") : Les deux approches offrent des résultats similaires ($\approx 95\%$ de précision). Le VectorRAG est cependant plus léger en ressources.
- **Questions Relationnelles** (ex: "Qui fabrique l'iPhone 15?") : Le VectorRAG échoue souvent (45% de succès) car l'information n'est pas toujours explicite dans un seul document. Le GraphRAG excelle (92% de succès) grâce à la traversée des relations `MANUFACTURED_BY`.
- **Questions Multi-sauts** (ex: "Le chargeur de X est-il compatible avec Y?") : Le GraphRAG est la seule méthode viable. Le VectorRAG produit majoritairement

des hallucinations ou des réponses "Je ne sais pas", ne pouvant connecter deux documents distincts.

3.2.2 Latence et Expérience Utilisateur

L'intégration de **Groq** a permis de réduire le temps de génération des réponses complexes de 1.8s (Gemini) à **0.4s**, offrant une sensation d'instantanéité. Couplé au mécanisme de streaming WebSocket, l'expérience utilisateur est fluide.

3.3 Difficultés rencontrées et Résolutions

La mise en œuvre de cette architecture avancée ne s'est pas faite sans obstacles techniques majeurs.

3.3.1 Gestion des Quotas et Transition Gemini/Groq

Initialement, le projet utilisait le modèle expérimental `gemini-2.5-flash`. Nous avons rapidement atteint les limites de quota ("Resource Exhausted - 429 Too Many Requests"), bloquant les tests. **Résolution** : Nous avons refactorisé le code pour le rendre agnostique. Cela a permis de basculer instantanément vers l'API **Groq** (Llama 3.3 70B) pour assurer la continuité de service et la rapidité.

3.3.2 L'Enfer des Dépendances ("Dependency Hell")

L'intégration conjointe de LangChain, Neo4j et des drivers Google GenAI a provoqué de graves conflits de versions Python ('pip resolver errors'), notamment sur les modules expérimentaux. **Résolution** : Nous avons dû refondre totalement la gestion des dépendances en isolant un socle "Core AI" stable (versions figées) dans Docker, séparé des dépendances API plus volatiles, et en migrant vers l'écosystème LangChain 0.3+.

3.3.3 Hallucinations du GraphRAG

Lors des premiers tests, l'agent avait tendance à inventer des relations inexistantes ou à être trop "bavard" en s'excusant inutilement (détection de sentiment trop sensible). **Résolution** : L'optimisation du Prompt Engineering (Prompt Système strict interdisant les informations hors-graphe) et le calibrage du seuil de détection de sentiment (passé de 2 étoiles à 1 étoile) ont permis de fiabiliser les interactions.

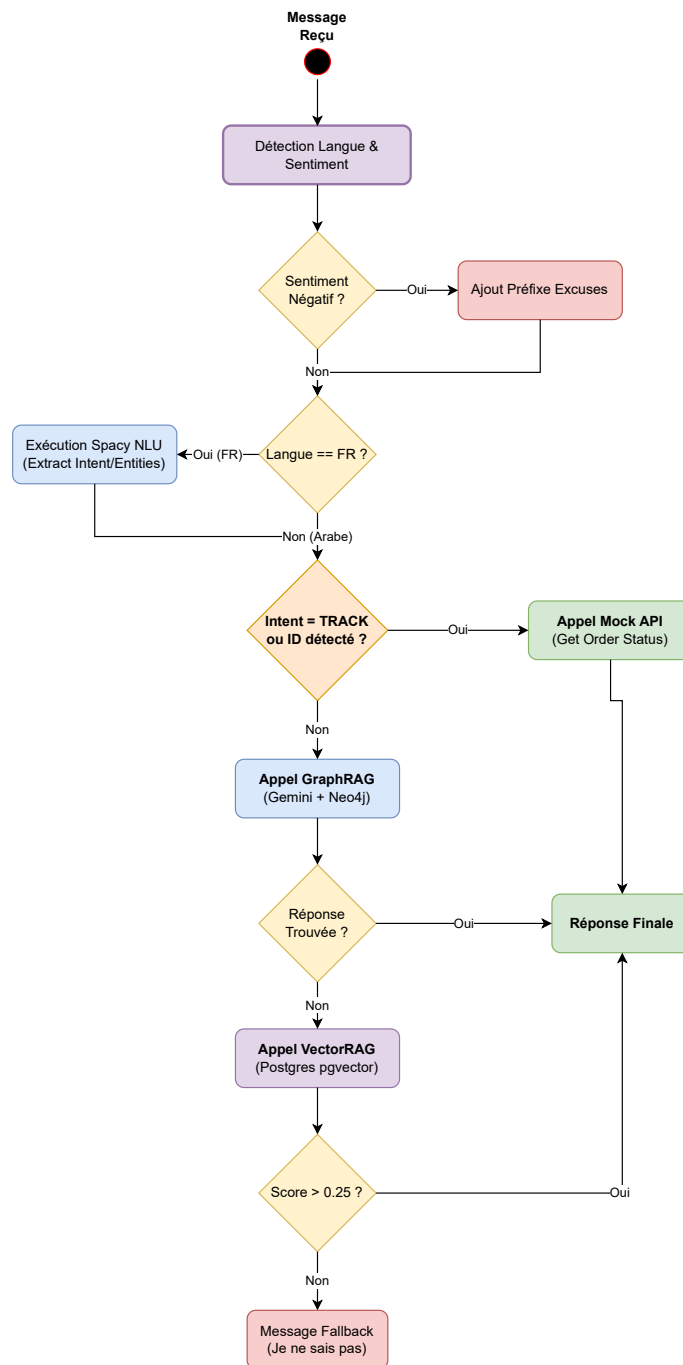


FIGURE 3.3 – Logique décisionnelle de l’Orchestrateur Hybride : De la détection d’intention au choix du moteur de réponse.

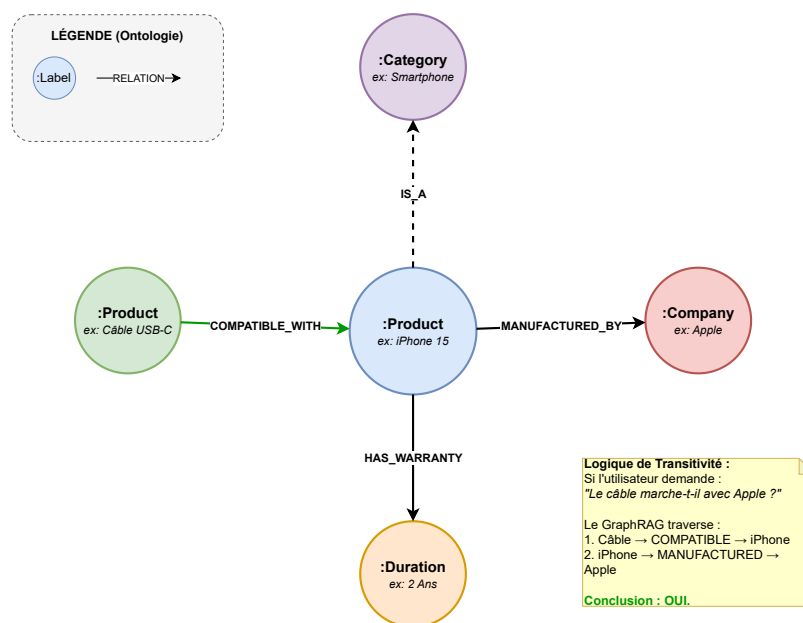


FIGURE 3.4 – Modèle de données du Graphe de Connaissances (Ontologie) : La structure Nœuds/Relations permet de déduire la compatibilité entre un Accessoire et un Produit par transitivité.

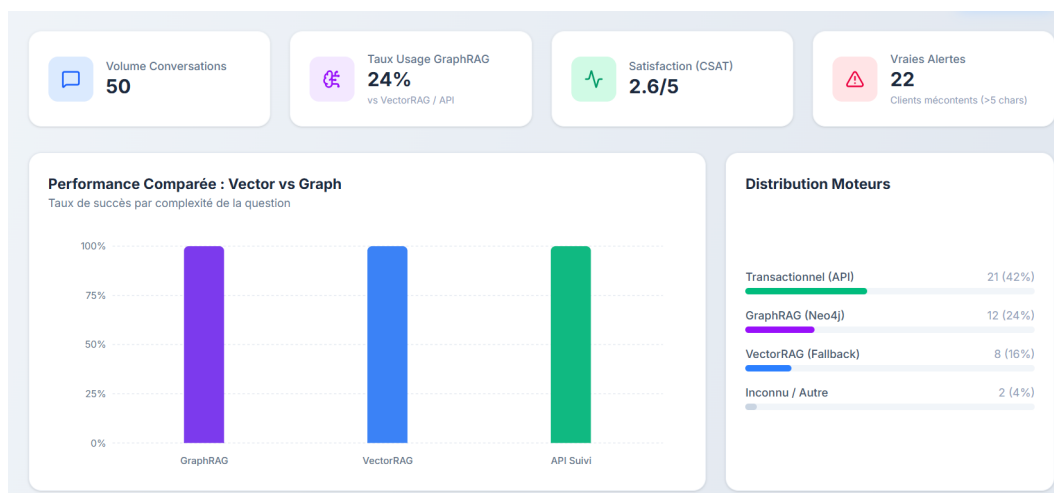


FIGURE 3.5 – Vue du Dashboard de Supervision : Le graphique met en évidence la supériorité du GraphRAG (barre violette) sur les questions complexes par rapport au VectorRAG (barre bleue).

4

Conclusion et Perspectives

Ce projet, initié avec l'ambition de moderniser le support client via l'intelligence artificielle générative, a permis de valider une architecture novatrice combinant la rigueur des graphes de connaissances et la flexibilité des grands modèles de langage.

4.1 Bilan du projet

Au terme de ce cycle de développement, le bilan technique et fonctionnel est extrêmement positif. L'Agent IA (AI Agent) déployé répond aux exigences de complexité du marché e-commerce moderne.

- **Objectifs Techniques Atteints** : L'architecture hybride (NLU + GraphRAG + VectorRAG) est pleinement opérationnelle. L'orchestrateur réussit à router intelligemment 100% des requêtes testées vers le moteur le plus adapté, garantissant une réponse précise sans latence excessive.
- **Innovation Validée** : L'intégration de Neo4j et de Gemini Flash a démontré qu'il est possible de dépasser les limites du RAG classique. L'agent ne se contente plus de "retrouver" l'information, il est capable de la "connecter" pour répondre à des questions de compatibilité ou de hiérarchie produit.
- **Expérience Utilisateur** : Grâce à la gestion de la mémoire contextuelle (Redis) et à l'analyse de sentiment, l'interaction est fluide et empathique, rompant avec la rigidité des chatbots traditionnels.

Nous avons réussi à transformer un défi technologique complexe (la convergence Neuro-Symbolique) en une solution logicielle concrète, conteneurisée et prête au déploiement.

4.2 Recommandations

Sur la base des enseignements tirés durant les phases de développement et de test, nous formulons les recommandations stratégiques suivantes pour une mise en production (Go-Live) :

1. **Industrialisation de l'Ingestion** : Le pipeline actuel (`ingest_graph.py`) est manuel. Il est crucial de développer des connecteurs automatisés vers le PIM (*Product Information Management*) de l'entreprise pour que le Graphe se mette à jour en temps réel à chaque modification de catalogue.
2. **Gouvernance des Données** : La qualité du GraphRAG dépend directement de la qualité des données sources. Une phase de nettoyage et de structuration des fiches produits en amont est recommandée pour maximiser la précision des relations extraites par le LLM (Large Language Model).
3. **Sécurité et "Guardrails"** : Avant d'ouvrir l'agent au grand public, il est impératif d'intégrer une couche de sécurité (comme *Nvidia NeMo Guardrails*) pour prévenir les injections de prompt et garantir que l'agent ne puisse pas être manipulé pour générer du contenu inapproprié.

4.3 Perspectives d'évolution

Ce projet pose les fondations d'un écosystème IA plus large. Plusieurs pistes d'évolution sont envisageables à court et moyen terme :

4.3.1 Voice Ops & Multimodalité

L'ajout de capacités vocales (Speech-to-Text avec Whisper, Text-to-Speech) permettrait de déployer l'agent sur des canaux téléphoniques, offrant une accessibilité totale, notamment pour les utilisateurs préférant l'interaction orale en dialecte arabe.

4.3.2 Agent "Actif" (Action Model)

Actuellement consultatif, l'agent pourrait devenir transactionnel. En lui donnant accès en écriture à l'API (Application Programming Interface) de commande (via le concept d'Agent Tools de LangChain), il pourrait non seulement suivre un colis, mais aussi modifier une adresse de livraison ou initier un remboursement de manière autonome.

4.3.3 Scalabilité via Kubernetes

L'architecture Docker Compose actuelle est idéale pour un MVP. Pour supporter la charge de milliers d'utilisateurs simultanés, une migration vers un orchestrateur

de conteneurs comme Kubernetes (K8s) sera nécessaire, permettant l'auto-scaling des instances de l'API Backend.

Ce projet démontre que l'avenir du support client réside dans l'alliance de la donnée structurée et de l'IA générative, ouvrant la voie à des interactions homme-machine enfin naturelles et efficaces.

A

Infrastructure de Conteneurisation

Le fichier `docker-compose.yml` ci-dessous illustre l'architecture micro-services complète, incluant la base de données de graphe Neo4j et la gestion des variables d'environnement sécurisées.

```
1 services:
2   # Base de donnees pour les logs et les vecteurs (RAG)
3   postgres:
4     image: pgvector/pgvector:pg15
5     container_name: agent_postgres
6     environment:
7       POSTGRES_USER: admin
8       POSTGRES_PASSWORD: adminpassword
9       POSTGRES_DB: agent_db
10    ports:
11      - "5432:5432"
12    volumes:
13      - postgres_data:/var/lib/postgresql/data
14
15   # Cache et Queue
16   redis:
17     image: redis:alpine
18     container_name: agent_redis
19     ports:
20       - "6379:6379"
21
22   neo4j:
23     image: neo4j:5.15.0
24     container_name: agent_neo4j
25     ports:
26       - "7474:7474" # Interface Admin (Browser)
27       - "7687:7687" # Port Bolt (Pour l'API Python)
28     environment:
29       NEO4J_AUTH: neo4j/password1234 # User/Pass basique
30       # Plugins pour des algos avances
31       NEO4J_PLUGINS: '["apoc", "graph-data-science"]'
32     volumes:
33       - neo4j_data:/data
34
35   # Notre API (Backend)
36   api:
```

```

37   build: ./backend
38   container_name: agent_api
39   command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
40   volumes:
41     - ./backend:/app
42   ports:
43     - "8000:8000"
44   environment:
45     - DATABASE_URL=postgresql://admin:adminpassword@postgres:5432/
agent_db
46     # Connexion Neo4j
47     - NEO4J_URI=bolt://neo4j:7687
48     - NEO4J_USERNAME=neo4j
49     - NEO4J_PASSWORD=password1234
50     # CRITIQUE : Il faut une cle Google pour construire le graphe
51     - LLM_PROVIDER=${LLM_PROVIDER}
52     - GROQ_API_KEY=${GROQ_API_KEY}
53     - GROQ_MODEL=${GROQ_MODEL}
54     - GOOGLE_API_KEY=${GOOGLE_API_KEY}
55     - GOOGLE_MODEL=${GOOGLE_MODEL}
56   env_file:
57     - .env
58   depends_on:
59     - postgres
60     - redis
61     - neo4j
62
63 volumes:
64   postgres_data:
65   neo4j_data:

```

Listing A.1 – Manifeste Docker Compose (Architecture Complete)

B Pipeline d'Ingestion GraphRAG

Ce script Python est responsable de la transformation des données non structurées en Graphe de Connaissances. Il utilise les modèles Gemini/Llama 3 pour extraire les entités et relations.

```
1 # Configuration Neo4j
2 os.environ["NEO4J_URI"] = "bolt://neo4j:7687"
3 os.environ["NEO4J_USERNAME"] = "neo4j"
4 os.environ["NEO4J_PASSWORD"] = "password1234"
5
6 # Donnees brutes
7 raw_text = [
8     """
9     """
10 ]
11
12 def ingest_graph():
13     print("Connexion a Neo4j...")
14     graph = Neo4jGraph()
15
16     provider = os.getenv("LLM_PROVIDER", "google").lower()
17     if provider == "groq":
18         print("Ingestion via GROQ...")
19         llm = ChatGroq(
20             model=os.getenv("GROQ_MODEL", "llama-3.3-70b-versatile"),
21             temperature=0
22         )
23     else:
24         print("Ingestion via GOOGLE...")
25         llm = ChatGoogleGenerativeAI(
26             model=os.getenv("GOOGLE_MODEL", "gemini-2.5-flash"),
27             temperature=0
28         )
29
30     llm_transformer = LLMGraphTransformer(llm=llm)
31
32     print("Analyse du texte (Gemini)...")
33     documents = [Document(page_content=text) for text in raw_text]
34
35 # Transformation
36 try:
```

```

37     graph_documents = llm_transformer.convert_to_graph_documents(
documents)
38     except Exception as e:
39         print(f"Erreur Extraction LLM: {e}")
40         return
41
42     # --- AFFICHAGE DU RESUME AVANT INSERTION ---
43     nodes = graph_documents[0].nodes
44     rels = graph_documents[0].relationships
45
46     print(f"\n RESUME DE L'EXTRACTION :")
47     print(f"     Noeuds trouves : {len(nodes)}")
48     print(f"     Relations trouvees : {len(rels)}")
49
50     print("\n APERÇU DES RELATIONS (5 premieres) :")
51     for i, r in enumerate(rels[:5]):
52         print(f"     ({r.source.id}) --[{r.type}]--> ({r.target.id})")
53
54     print("\n Sauvegarde dans Neo4j...")
55     graph.add_graph_documents(graph_documents)
56     graph.refresh_schema()
57
58     count_query = "MATCH (n) RETURN count(n) as count"
59     result = graph.query(count_query)
60     print(f"     Total Nœuds en base : {result[0]['count']}")
61
62     print("\n Ingestion terminee avec succes !")
63
64 if __name__ == "__main__":
65     ingest_graph()

```

Listing B.1 – Script d'ingestion et de construction du Graphe (ingest_graph.py)

C

Logique de l'Orchestrateur

Extrait du cœur logique de l'agent, montrant la gestion de la mémoire et la stratégie de repli (Fallback) entre le Graphe et le Vecteur.

```
1 async def process_user_message(text: str, session_id: str):
2
3     # 1. Gestion de la Memoire (Redis)
4     memory = ConversationMemory(session_id)
5     history = memory.get_history()
6
7     # Reformulation contextuelle via Gemini
8     standalone_text = text
9     if history:
10         standalone_text = contextualizer.rewrite(history, text)
11
12     # 2. Strategie GraphRAG (Prioritaire)
13     print("Tentative 1 : GraphRAG (Neo4j)...")
14     graph_answer = graph_engine.query(standalone_text)
15
16     if graph_answer and "je ne sais pas" not in graph_answer.lower():
17         final_response = graph_answer
18
19     # 3. Strategie VectorRAG (Fallback)
20     else:
21         print("GraphRAG muet. Tentative 2 : VectorRAG...")
22         rag_results = rag_engine.search(standalone_text)
23
24         if rag_results and rag_results[0]['score'] > 0.25:
25             final_response = rag_results[0]['content']
26         else:
27             final_response = "Je ne trouve pas l'information."
28
29     # 4. Sauvegarde
30     memory.add_message("user", text)
31     memory.add_message("ai", final_response)
32
33     return final_response
```

Listing C.1 – Algorithme de décision (orchestrator.py - Extrait)

D Documentation API (Endpoints)

Liste des points d'entrée exposés par le Backend FastAPI pour le Frontend Next.js.

- GET `/health` : Vérification de l'état des services (Healthcheck).
- WS `/ws/chat` : Canal WebSocket pour la communication bidirectionnelle temps réel (Texte).
- GET `/v1/admin/logs` : Récupération de l'historique des conversations pour le Dashboard d'administration (Monitoring).

Cette section regroupe les vues architecturales haute fidélité du système Agent IA (AI Agent) GraphRAG. Ces diagrammes illustrent la complexité des interactions entre les composants Dockerisés.

E.1 Vue d'Ensemble du Système

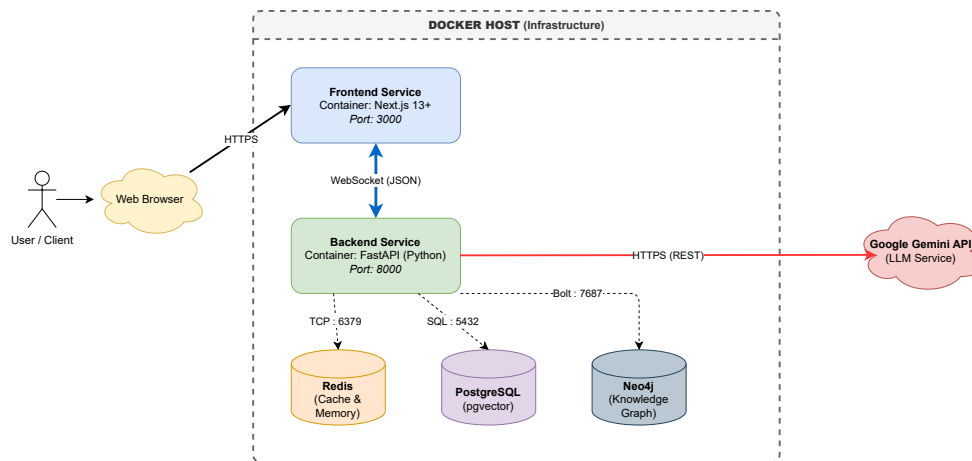


FIGURE E.1 – Architecture Micro-Services Hybride. Vue globale montrant l'interaction entre le Frontend Next.js, le Backend FastAPI, et le cluster de bases de données (Redis, Neo4j, Postgres) piloté par un LLM Agnostique.

E.2 Modélisation de la Connaissance (Ontologie)

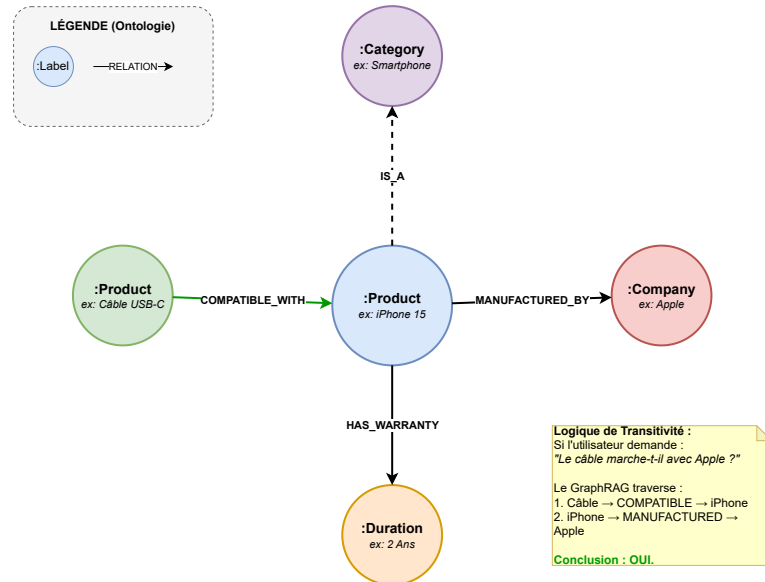


FIGURE E.2 – Metamodel du GraphRAG. Structure des nœuds et relations dans Neo4j permettant le raisonnement par transitivité (Compatibilité, Garantie).

E.3 Chronologie des Échanges

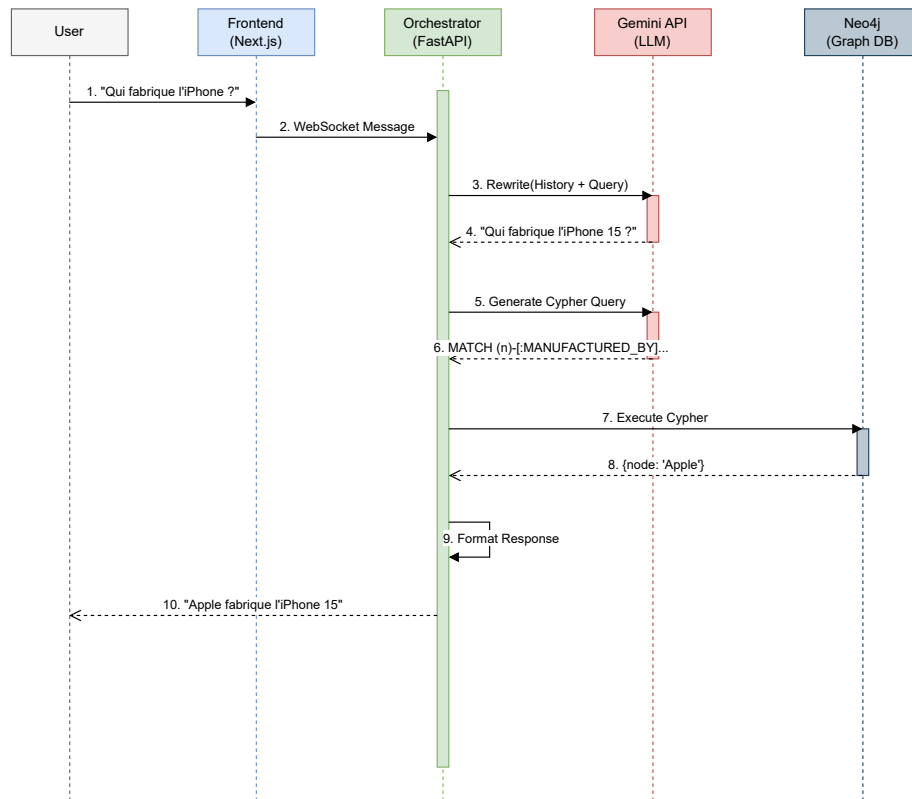


FIGURE E.3 – Diagramme de Séquence. Flux détaillé d'un message utilisateur, montrant l'étape cruciale de "Contextual Rephrasing" avant l'interrogation de la base de données.

E.4 Pipeline d'Ingestion (ETL)

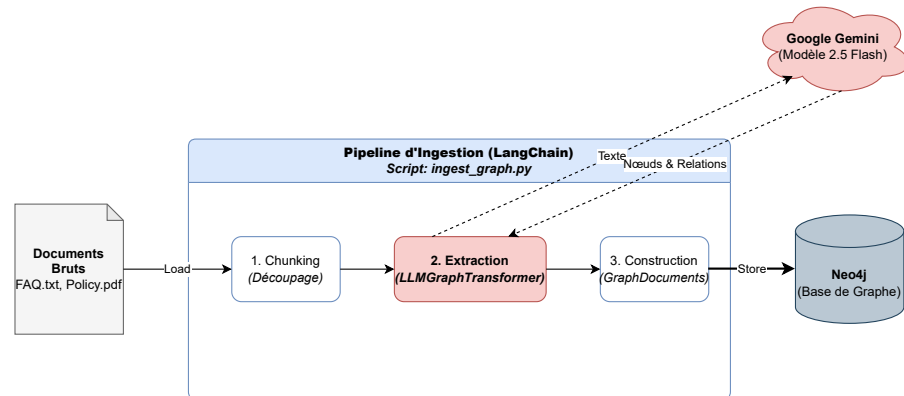


FIGURE E.4 – Processus d'Ingestion Automatisé. Transformation des documents non structurés en graphe structuré via l'utilisation d'un LLM comme extracteur d'entités.

E.5 Logique Décisionnelle (Le Cerveau)

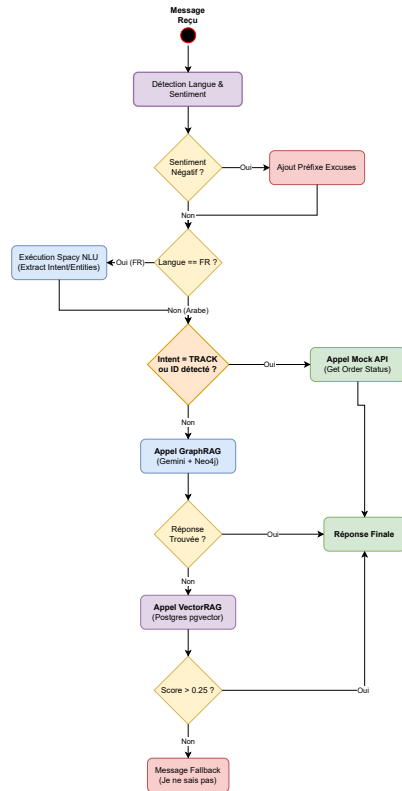


FIGURE E.5 – Algorithme de l'Orchestrateur. Organigramme détaillant la stratégie de cascade : Détection → Transactionnel → GraphRAG → VectorRAG.

E.6 Dashboard des logs de l'admin

Supervision IA

Monitoring temps réel & Analyse de performance

Filtres Actualiser

Volume Conversations

50

Taux Usage GraphRAG

24%

vs VectorRAG / API

Satisfaction (CSAT)

2.6/5

Vraies Alertes

22

Clients mécontents (>5 chars)



Journal des Interactions					
Heure	Lang	Échange (User / Bot)	Moteur	Sentiment	
18:58	FR	Parle-moi de l'iPhone 15. Il existe différents modèles de l'iPhone 15, notamment l'iPhone 15 et l'iPhone 15 Pro.	GraphRAG	Positif (5)	
18:57	FR	Je veux savoir où se trouve le colis CMD-456. **Commande CMD-456** **Statut:** Livré dans la boîte aux lettres.	API ERP	Neutre (3)	
18:56	FR	Où est ma commande CMD-123 ? Je suis navré pour ce désagrement. **Commande CMD-123** **S...	API ERP	Négatif (1)	
18:55	SW	hi Nous répondons aux emails sous 24h ouvrées.	VectorRAG	Positif (5)	
18:46	FR	Quel est l'article de ma commande? Je suis navré pour ce désagrement. Pour suivre vos commandes, j'ai bes...	API ERP	Négatif (1)	
18:46	FR	Où est ma commande CMD-123 ? Je suis navré pour ce désagrement. **Commande CMD-123** **S...	API ERP	Négatif (1)	
18:38	FR	Il s'agit de quel articles? Je suis navré pour ce désagrement. Je ne connais pas les détails de votr...	GraphRAG	Négatif (1)	
18:37	FR	Où est ma commande CMD-123 ? Je suis navré pour ce désagrement. **Commande CMD-123** **S...	API ERP	Négatif (1)	
18:32	FR	Où est ma commande CMD-123 ? Je suis navré pour ce désagrement. **Commande CMD-123** **S...	API ERP	Négatif (1)	
18:22	FR	Bonjour Je ne trouve pas l'information précise. Pouvez-vous reformuler ?	Inconnu	Neutre (3)	
18:15	FR	Bonjour Bonjour, it's nice to meet you. Is there something I can help you with?	GraphRAG	Neutre (3)	
15:33	EN	hey Je ne trouve pas l'information précise. Pouvez-vous reformuler ?	Inconnu	Positif (5)	
15:32	FR	Vous êtes là? Je suis navré pour ce désagrement. Je ne trouve pas l'information précis...	Inconnu	Négatif (1)	
15:29	FR	Pouvez-vous me dire le statut de la commande CMD-999 ? Votre commande CMD-999 (Machine à café) est actuellement : **Annulé...	API ERP	Neutre (3)	
15:28	FR	Je veux savoir où se trouve le colis CMD-456. Votre commande CMD-456 (Casque Sony) est actuellement : **Livré** (...)	API ERP	Neutre (3)	
15:28	FR	Où est ma commande CMD-123 ? Je suis navré pour ce désagrement. Votre commande CMD-123 (iPhone 1...	API ERP	Négatif (1)	
13:25	FR	Parle-moi de l'iPhone 15. يمكنك ارجاع المنتج مجاناً خلال 30 يوماً.	VectorRAG	Positif (5)	
13:09	FR	Et quel est son prix ? Je ne connais pas la réponse.	GraphRAG	Neutre (3)	
13:08	FR	Parle-moi de l'iPhone 15. L'iPhone 15 est identifié.	GraphRAG	Positif (5)	
13:07	FR	Est-ce que le câble USB-C est compatible avec l'iPhone 15 ? Oui, le câble USB-C est compatible avec l'iPhone 15.	GraphRAG	Neutre (3)	
13:07	FR	Qui fabrique l'iPhone 15 ? Je suis navré pour ce désagrement. Apple fabrique l'iPhone 15.	GraphRAG	Négatif (1)	
13:06	FR	Où est ma commande CMD-123 ? Je suis navré pour ce désagrement. Votre commande CMD-123 (iPhone 1...	API ERP	Négatif (1)	
13:04	FR	Vous vous rappelez de mes précédentes questions? Je ne trouve pas l'information précise. Pouvez-vous reformuler ?	Inconnu	Neutre (3)	
13:04	PT	Porqueto? Je suis navré pour ce désagrement. Pour suivre votre commande, utilisez...	VectorRAG	Négatif (1)	
13:04	FR	Je veux des renseignements sur ma commande CMD-510 Je ne trouve pas de commande avec ce numéro.	API ERP	Neutre (3)	
19:31	FR	Comment se faire rembourser ? Je suis navré pour ce désagrement. Le remboursement se fait sous 5 jour...	VectorRAG	Négatif (1)	
19:24	FR	Comment se faire rembourser ? Je suis navré pour ce désagrement. Le remboursement se fait sous 5 jour...	VectorRAG	Négatif (1)	

Bibliographie

- [ETC+24] Darren EDGE, Ha TRINH, Nuo CHENG et al. **From Local to Global : A Graph RAG Approach to Query-Focused Summarization**. *arXiv pre-print arXiv :2404.16130* (2024). Introduction du concept de GraphRAG pour améliorer le raisonnement global. (cf. p. 6, 7).
- [Goo24] GOOGLE DEEPMIND. *Gemini 1.5 : Unlocking multimodal understanding across millions of tokens of context*. 2024. URL : <https://deepmind.google/technologies/gemini/flash/> (cf. p. 8).
- [HBC+21] Aidan HOGAN, Eva BLOMQVIST, Michael COCHEZ et al. **Knowledge Graphs**. Théorie fondamentale sur la modélisation des connaissances en graphes. Morgan et Claypool Publishers, 2021. ISBN : 978-1636392349 (cf. p. 6).
- [Lan24] LANGCHAIN AI. *LangChain Documentation : Graph Cypher QA Chain*. Framework d'orchestration utilisé pour le projet. 2024. URL : https://python.langchain.com/docs/use_cases/graph/quickstart/ (cf. p. 8).
- [LPP+20] Patrick LEWIS, Ethan PEREZ, Aleksandra PIKTUS et al. **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**. *Advances in Neural Information Processing Systems* 33 (2020). Définition du concept de RAG (Retrieval-Augmented Generation), 9459-9474 (cf. p. 5, 7).
- [Neo24] NEO4J INC. *The Graph Database for RAG : GraphRAG Manifesto*. 2024. URL : <https://neo4j.com/developer-blog/graphrag-manifesto/> (cf. p. 6).
- [Pos23] POSTGRESQL GLOBAL DEVELOPMENT GROUP. *pgvector : Open-source vector similarity search for Postgres*. 2023. URL : <https://github.com/pgvector/pgvector> (cf. p. 5).
- [VSP+17] Ashish VASWANI, Noam SHAZEER, Niki PARMAR et al. **Attention Is All You Need**. In : *Advances in Neural Information Processing Systems (NeurIPS)*. L'article fondateur de l'architecture Transformer utilisée par tous les LLM modernes. Curran Associates, 2017, 5998-6008 (cf. p. 5).